

Arquitectura U-Net para imágenes satelitales

Aprendizaje Profundo

Gustavo de Jesús Escobar Mata

Facultad de Ciencias Físico Matemáticas

22 de abril de 2026

- Introducción
- Determinación del problema
- Datos
- Arquitectura
- Resultados y discusiones
- Investigación futura y limitaciones

El crecimiento acelerado de las ciudades genera una demanda urgente de información geoespacial precisa y actualizada. La clasificación manual del uso y cobertura del suelo (**LULC**) es costosa, lenta y difícil de escalar a nivel global.

El **aprendizaje profundo** ofrece una alternativa eficiente: modelos de segmentación semántica capaces de clasificar cada píxel de una imagen satelital de forma automática, a bajo costo y con alta precisión.

En este proyecto se propone y evalúa una arquitectura **U-Net con encoder EfficientNet-B3** entrenada sobre imágenes multiespectrales **Sentinel-2** para clasificar el uso de suelo en 4 categorías (Urbano, Vegetación, Agua y Suelo árido) en 10 ciudades de 6 continentes. El modelo se integra en **UrbanAI**, una plataforma que combina el modelo con un agente de IA para democratizar el análisis territorial.

Problema:

Cartografía manual lenta y costosa

Solución:

U-Net + Sentinel-2 automatizado

Resultado:

mIoU = 0.8239 (Excelente)

Problema de negocio: El crecimiento acelerado de las ciudades genera una demanda creciente de información precisa y actualizada sobre el uso y cobertura del suelo (Land Use / Land Cover, LULC). Gobiernos, institutos de planeación y organismos de desarrollo urbano requieren conocer, con resolución espacial detallada, cómo se distribuyen las zonas construidas, la vegetación, los cuerpos de agua y el suelo árido dentro de sus territorios; sin embargo, los métodos tradicionales de cartografía (basados en trabajo de campo o fotointerpretación manual) son costosos, lentos y difíciles de escalar a múltiples ciudades simultáneamente.

Para este proyecto se seleccionaron imágenes satelitales de las ciudades mostradas en la Tabla 1, considerando la diversidad de características climáticas.

Tabla 1: Ciudades incluidas en el conjunto de datos

Ciudad	País	Característica climática
Amsterdam	Países Bajos	Oceánico, alta vegetación
Bangkok	Tailandia	Tropical húmedo, ríos
Bogotá	Colombia	Altiplano, zona montañosa
Dubai	Emiratos Árabes	Árido, expansión urbana rápida
Houston	EE.UU.	Subtropical, ciudad extensa
Madrid	España	Mediterráneo continental
Ciudad de México	México	Altiplano, megaciudad
Monterrey	México	Semiárido, montañoso
Mumbai	India	Monzónico, costera
Nairobi	Kenia	Tropical de altitud

Para cada ciudad de la Tabla 1, se descargaron imágenes de las cuatro estaciones de 2023 con filtro de nubosidad máxima del 10 %. Se utilizó Sentinel-2, misión de la ESA dentro del programa Copernicus. Los parámetros empleados se detallan en la Tabla 2.

Tabla 2: Parámetros de Sentinel-2

Parámetro	Valor
Resolución espacial	10 m/pixel (bandas RGB + NIR + SWIR)
Resolución temporal	5 días (revisita)
Bandas utilizadas	B02, B03, B04, B08, B11, B12
Tipo de dato	Reflectancia de superficie (BOA - L2A)
Formato de descarga	GeoTIFF (.jp2 interno)
Rango de valores	0.0 – 1.0 (reflectancia normalizada)
Cobertura de nubes	Se descarta si supera el 10 %
Proveedor de acceso	Copernicus Dataspace (OAuth2)

Las cuatro categorías utilizadas en el proyecto se muestran en la Tabla 3.

Tabla 3: Clases de uso/cobertura del suelo (LULC)

ID	Clase	Descripción
0	Urbano / Construido	Edificios, calles, infraestructura impermeable
1	Vegetación / Bosque	Áreas verdes, parques, bosques, cultivos
2	Agua	Cuerpos de agua, ríos, lagos, costas
3	Suelo desnudo / Árido	Desiertos, zonas sin cobertura vegetal

Las imágenes se obtienen mediante la API de **Copernicus Dataspace** (OAuth2). El preprocesamiento consistió en los siguientes pasos:

- 1 Definición del *bounding box* por ciudad (lon_min , lat_min , lon_max , lat_max).
- 2 Descarga automática de imágenes **Sentinel-2 L2A** con nubosidad $< 10\%$, seleccionando la de menor nubosidad por ciudad y estación.
- 3 Cálculo y descarga de *tiles* **ESA WorldCover 2021** (grilla 3×3 , 12 tiles).
- 4 Reproyección y remuestreo de B11/B12 a 10 m mediante interpolación bilineal.
- 5 Apilado de 6 bandas y normalización al rango $[0,0, 1,0]$.
- 6 Reproyección de máscaras WorldCover al CRS de Sentinel-2 (vecino más cercano).
- 7 Remapeo de clases WorldCover a 4 categorías LULC; píxeles no asignados marcados como 255 (ignorados en loss).
- 8 Extracción de *patches* 256×256 px con stride 128 px; se descartan patches con $> 50\%$ sin datos o media $< 0,01$.

La Tabla 4 muestra la distribución de patches por ciudad, obtenidos a partir de imágenes Sentinel-2 de las cuatro estaciones de 2023 con la cual se entreno el modelo.

Tabla 4: Patches por ciudad

Ciudad	País	Continente	Patches
Bangkok	Tailandia	Asia	28,224
Monterrey	México	Latinoamérica	28,224
Madrid	España	Europa	21,146
Nairobi	Kenia	África	15,447
Dubái	Emiratos Árabes	Medio Oriente	15,950
Ámsterdam	Países Bajos	Europa	17,557
Ciudad de México	México	Latinoamérica	11,796
Bogotá	Colombia	Latinoamérica	7,056
Mumbai	India	Asia	4,668
Houston	EE.UU.	Norteamérica	864
Total			150,932

El conjunto de datos se dividió en 70 % para entrenamiento, 15 % para validación y 15 % para evaluación. La Tabla 5 resume la configuración de hiperparámetros empleada durante el entrenamiento del modelo.

Tabla 5: Configuración de entrenamiento

Parámetro	Valor
Arquitectura	U-Net + EfficientNet-B3
Optimizer	AdamW (10^{-4})
LR Scheduler	Cosine Annealing
Loss function	CE + Dice (50/50)
Batch size	16
Épocas	50
Early Stopping	Patience = 10

Parámetro	Valor
Total de patches	105,652
Tamaño de patch	256×256 px
Bandas de entrada	6 (B02-B12)
Ciudades incluidas	10
Semilla	42
Stride	128 px (50 %)
Estaciones	4

U-Net es una CNN diseñada originalmente para segmentación semántica de imágenes médicas (Ronneberger et al., 2015). Su estructura en forma de “U” consiste en:

- **Encoder:** extrae features jerárquicas mediante bloques convolucionales y capas de pooling, reduciendo progresivamente la resolución espacial.
- **Decoder:** reconstruye la resolución espacial original mediante upsampling y convoluciones transpuestas.
- **Skip connections:** conexiones simétricas encoder–decoder que preservan información espacial de alta resolución.

EfficientNet-B3 reemplaza el encoder estándar mediante **compound scaling** (Tan & Le, 2019), que escala simultáneamente profundidad, anchura y resolución de forma balanceada. La integración se realizó con `segmentation-models-pytorch`.

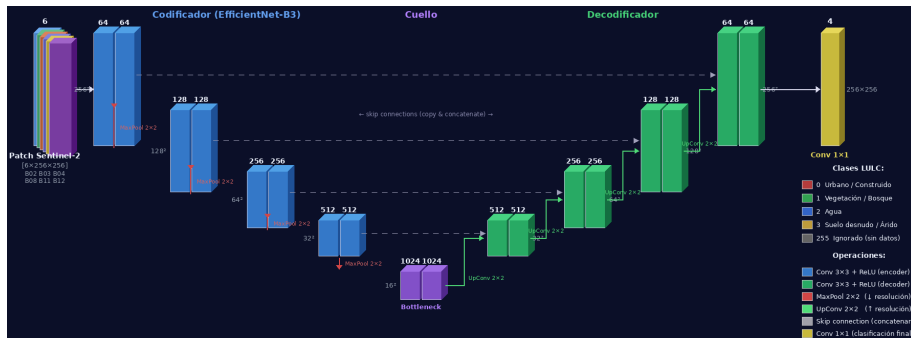


Figura 1: Arquitectura U-Net con encoder EfficientNet-B3

Función de pérdida (CrossEntropy estándar de PyTorch):

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=0}^{C-1} y_{ic} \cdot \log \hat{p}_{ic} \quad (1)$$

donde y_{ic} es la etiqueta one-hot del píxel i para la clase c , y $\hat{p}_{ic}$ la probabilidad predicha. Se combina con Dice Loss (50/50).

Métrica principal — **mIoU** (mean Intersection over Union):

$$\text{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}, \quad \text{mIoU} = \frac{1}{C} \sum_{c=0}^{C-1} \text{IoU}_c \quad (2)$$

Interpretación del mIoU:

- $> 0,80$ → **Excelente**
- $0,65-0,80$ → **Bueno**
- $0,50-0,65$ → **Aceptable**
- $< 0,50$ → **Insuficiente**

Métricas adicionales monitoreadas:

- **Pixel Accuracy:** fracción de píxeles correctamente clasificados (menos informativa ante desbalance de clases).
- **Loss de validación:** para detectar sobreajuste.
- **Loss de entrenamiento:** para monitorear convergencia.

El modelo fue entrenado e implementado en **Google Cloud Platform (GCP)**, aprovechando sus servicios de almacenamiento y cómputo en la nube para garantizar escalabilidad y disponibilidad. Las Tablas 6 y 7 resumen la configuración empleada.

Tabla 6: Google Cloud Storage (GCS)

Parámetro	Valor
Clase de almacenamiento	Standard
Región	us-central1
Control de acceso	Uniform
Contenido	Imágenes + modelo

Tabla 7: Virtual Machine (VM)

Parámetro	Valor
vCPU	2
RAM	4 GB
Disco SSD	400 GB
Región	us-central1
Timeout (request)	120 s
Concurrencia	1 request
Autoescalado mín.	0 instancias
Autoescalado máx.	3 instancias

En el proyecto se implementó un agente basado en herramientas siguiendo el paradigma **ReAct** (*Reasoning + Acting*): el agente razona sobre la pregunta del usuario, selecciona la herramienta adecuada, observa el resultado e itera.

Tabla 8: Componentes del agente

Componente	Implementación
LLM (cerebro)	Claude Sonnet
Orquestador	<code>create_react_agent</code>
Memoria de sesión	Historial de mensajes
Contexto base	SYSTEM_PROMPT
Framework	LangChain + LangGraph

Tabla 9: Ciclo ReAct del agente

Paso	Fase	Acción
1	Percepción	Recibe pregunta del usuario
2	Razonamiento	Decide qué tool invocar
3	Actuación	Ejecuta la herramienta
4	Observación	Analiza el resultado
5	Respuesta	Genera respuesta final

El agente dispone de **4 herramientas** que actúan como sus actuadores [9]. La selección de herramienta en cada ciclo constituye una **heurística implícita**: el LLM evalúa qué acción maximiza la calidad de respuesta dado el contexto, sin explorar exhaustivamente todas las combinaciones posibles.

Tabla 10: Herramientas (actuadores) del agente

Tool	Función
classify_city	Clasifica uso de suelo con U-Net
get_city_stats	Estadísticas del dataset por ciudad
list_cities	Lista las 10 ciudades disponibles
search_urban_docs	Búsqueda semántica en documentos (RAG)

Tabla 11: Heurística:
temperature=0

Parámetro	Efecto
temperature=0	Respuestas deterministas
max_tokens=4096	Límite de razonamiento
Idioma forzado	Español
Redondeo	1 decimal

El agente no conoce el problema completo de antemano, sino que actúa a medida que recibe percepciones. El modo `interactive_chat` implementa exactamente este paradigma. El componente **RAG** (*Retrieval-Augmented Generation*) extiende el conocimiento del agente con documentos externos en tiempo de inferencia.

Tabla 12: En línea vs. fuera de línea

Característica	En línea	Fuera de línea
Conoce todo el problema	No	Sí
Actúa mientras percibe	Sí	No
Implementación	<code>interactive_chat</code>	Entrenamiento U-Net

Tabla 13: Enfoques de IA

Enfoque	Componente
Actuar racionalmente	Agente ReAct (LLM)
Pensar racionalmente	RAG + razonamiento
Actuar humanamente	Respuestas en lenguaje natural
Aprender	U-Net entrenado (Deep Learning)

El modelo se entreno para una muestra de 22,639 patches clasificando cada pixel en 4 clases. El test loss obtenido para este conjunto fue de 0.2019 como se muestran en la Tabla 14. Se obtiene **mIoU = 0.82**, indicando clasificación excelente. La Tabla 15 desglosa las métricas por clase.

Tabla 14: Resultados de evaluación del modelo

Métrica	Valor
Número de clases	4
Muestras de prueba	22,639
Test Loss	0.2019
mIoU	0.8239
Mean F1	0.9000
Exactitud global (pixel)	92.16 %
Tiempo (segundos)	1,201.89

Tabla 15: Resultados por clase

Clase	IoU	Precisión	Recall	F1
Urbano / Construido	0.7536	0.8472	0.8691	0.8576
Vegetación / Bosque	0.8804	0.9367	0.9355	0.9360
Agua	0.8686	0.9322	0.9162	0.9231
Suelo desnudo / Árido	0.7931	0.8902	0.8767	0.8832

Se desarrolló **UrbanAI**, plataforma que implementa el modelo junto con un agente de IA mediante la API de Anthropic (Figura 5). Permite obtener, en minutos y para cualquier ciudad, segmentación semántica de alta precisión con estadísticas de distribución por clase (Figuras 2 y 3).

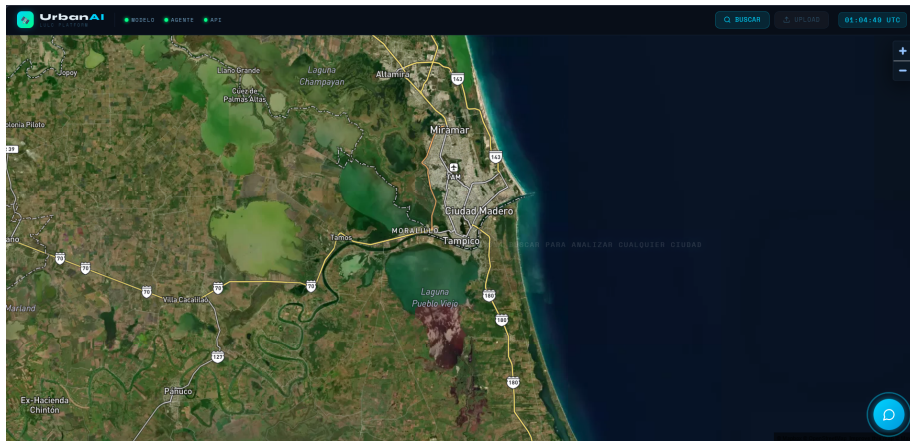


Figura 2: Análisis de clasificación en Tampico, Tamaulipas, México

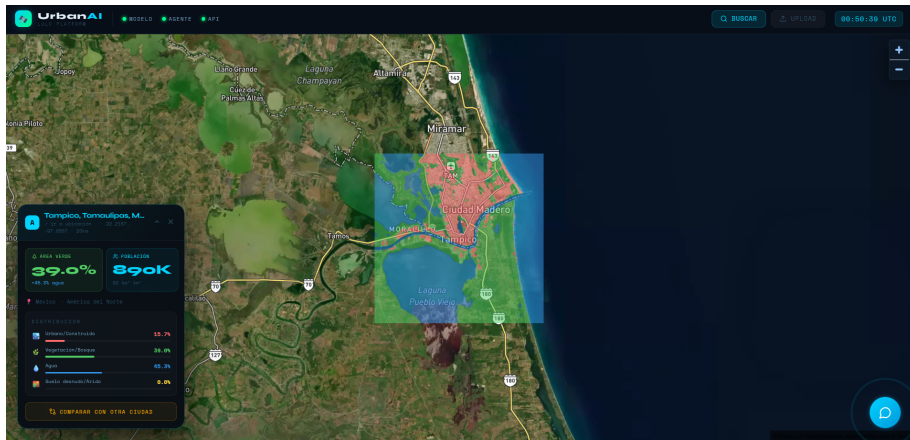


Figura 3: Resultado de clasificación en Tampico, México

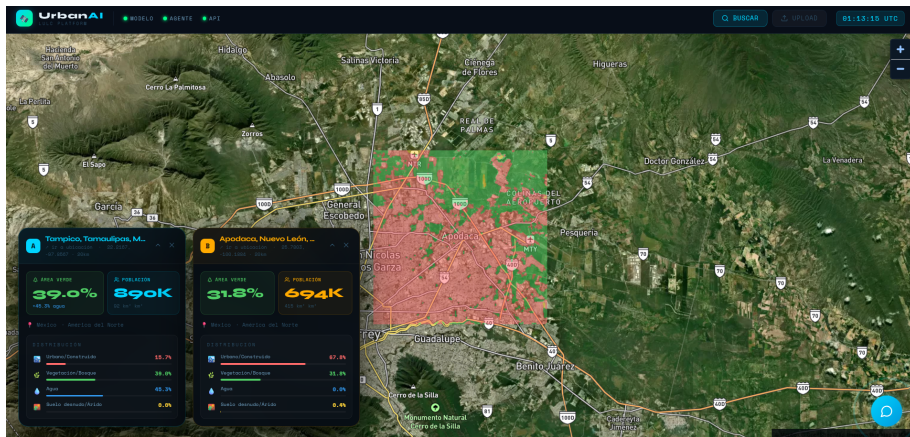


Figura 4: Resultado de clasificación en Apodaca, Nuevo León, México

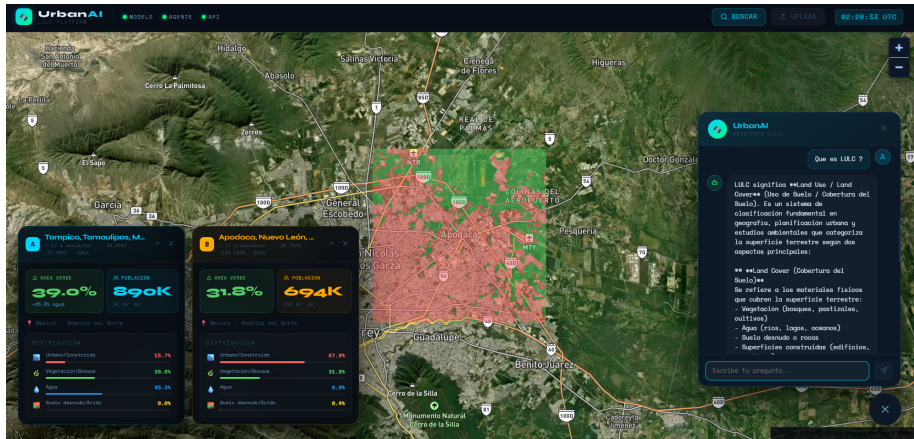


Figura 5: Agente de IA implementado mediante la API de Anthropic

UrbanAI permite el monitoreo de expansión urbana no planificada, la detección de pérdida de áreas verdes, la evaluación de superficies impermeables que agravan inundaciones, y el seguimiento del cambio de uso de suelo a lo largo del tiempo, de manera eficiente respecto a los métodos tradicionales de cartografía.

- La clase **Industrial** presenta $\text{IoU} = 0,0$ en el modelo actual. Se requiere reentrenamiento con `class_weight` para corregir el desbalance de clases.
- Reentrenamiento con pesos inversamente proporcionales a la frecuencia de cada clase.
- Análisis de predicción de niveles de contaminación mediante Sentinel-5P.
- Despliegue de la aplicación en Cloud Run de GCP.

- [1] Copernicus Open Access Hub. Copernicus open access hub. <https://scihub.copernicus.eu/>. Accessed: 2026-04-12.
- [2] European Space Agency. *Sentinel-2 User Handbook*. ESA, 2024.
- [3] European Space Agency. *Sentinel-5P Product User Manual — TROPOMI NO2*. ESA, 2024.
- [4] Google LLC. Google cloud platform. <https://cloud.google.com>, 2024. Accessed: 2026-04-16.
- [5] LangChain. Langchain documentation. <https://python.langchain.com/>. Accessed: 2026-04-12.
- [6] Mapbox. Mapbox gl js documentation. <https://docs.mapbox.com/mapbox-gl-js/>. Accessed: 2026-04-12.
- [7] Qubvel. segmentation-models-pytorch. https://github.com/qubvel/segmentation_models.pytorch. Accessed: 2026-04-12.
- [8] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI*, 2015.
- [9] Stuart Russell and Peter Norvig. Inteligencia artificial: Un enfoque moderno. In *Pearson*, 2022.
- [10] Sentinel Hub. Sentinel hub documentation. <https://docs.sentinel-hub.com/>. Accessed: 2026-04-12.
- [11] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.

Gracias por tu atención

Gustavo de Jesús Escobar Mata
gustavo.escobarma@uanl.edu.mx